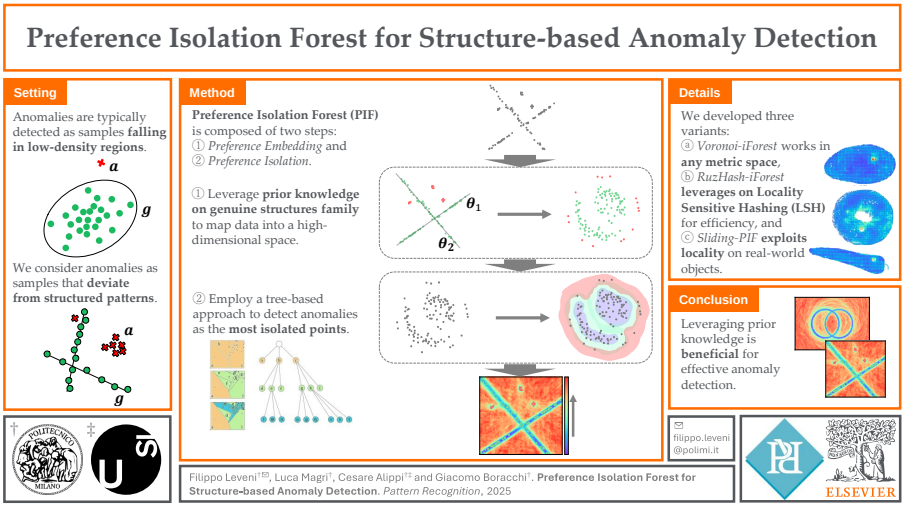
arXiv:2505.10876v2 [cs.LG] 17 Sep 2025

Graphical Abstract

Preference Isolation Forest for Structure-based Anomaly Detection

Filippo Leveni, Luca Magri, Cesare Alippi, Giacomo Boracchi



Highlights

Preference Isolation Forest for Structure-based Anomaly Detection

Filippo Leveni, Luca Magri, Cesare Alippi, Giacomo Boracchi

- We propose a general framework to detect anomalies with respect to parametric structured patterns.
- We enhance separability between genuine and anomalous data via embedding into a high-dimensional preference space, leveraging on prior knowledge on genuine data.
- We detect anomalies as the most isolated in the embedding, by proposing tailored isolation-based methods able to exploit the correct distance measures in the preference space.
- We propose a sliding window extension able to deal with unknown but smooth patterns by leveraging on locality priors.
- Extensive experiments on synthetic and real-world data demonstrate the effectiveness of structure-based anomaly detection.

Preference Isolation Forest for Structure-based Anomaly Detection

Filippo Leveni[a,*], Luca Magri[a], Cesare Alippi[a,b], Giacomo Boracchi[a]

[a] *Department of Electronics, Information and Bioengineering, Politecnico di Milano, Piazza Leonardo da Vinci, 32, Milano, 20133, Italy*
[b] *Faculty of Informatics, Università della Svizzera Italiana, Via Giuseppe Buffi, 13, Lugano, 6900, Switzerland*

Abstract

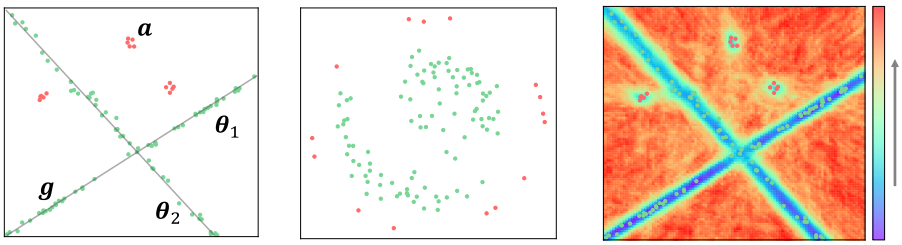
We address the problem of detecting anomalies as samples that do not conform to structured patterns represented by low-dimensional manifolds. To this end, we conceive a general anomaly detection framework called Preference Isolation Forest (PIF), that combines the benefits of adaptive isolation-based methods with the flexibility of preference embedding. The key intuition is to embed the data into a high-dimensional preference space by fitting low-dimensional manifolds, and to identify anomalies as isolated points. We propose three isolation approaches to identify anomalies: *i*) Voronoi-iForest, the most general solution, *ii*) RuzHash-iForest, that avoids explicit computation of distances via Local Sensitive Hashing, and *iii*) Sliding-PIF, that leverages a locality prior to improve efficiency and effectiveness.

Keywords: structure-based anomaly detection, isolation-based anomaly detection

1. Introduction

Anomaly detection (AD) deals with the problem of identifying data that do not conform to an expected behavior [1]. In the statistical and data-mining literature, *anomalies* are typically detected as samples falling in low-density regions [1]. However, in many real-world scenarios, genuine data lie on low-dimensional manifolds and anomalies are best characterized in terms of non-conformity to those structures. For example, in 3D scans [2] anomalies can be recognized as defects that deviate from the smooth surfaces that characterized the object. This remains true regardless of their local density, which may vary depending on the distance from the sensing device. In such a situation, the density analysis in the ambient space becomes misleading: scanning artifacts may lie in a dense region but still be structurally inconsistent with the surrounding data.

*Corresponding author.
Email addresses: filippo.leveni@polimi.it (Filippo Leveni), luca.magri@polimi.it (Luca Magri), cesare.alippi@polimi.it (Cesare Alippi), giacomo.boracchi@polimi.it (Giacomo Boracchi)



(a) Input data $X = G \cup A$. (b) Preference space $\mathcal{P}$. (c) Anomaly scores α.

Figure 1: PIF detects anomalies that deviate from structures. (a) Genuine points G, in green, described by two lines $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$, and anomalies A in red. (b) Preference Embedding maps X to a high-dimensional space where anomalies result in isolated points (visualized via MultiDimensional Scaling). (c) Anomaly scores $\alpha(\cdot)$ computed via Preference Isolation.

Similarly, in intrusion detection [3], where the purpose is to identify faces of unauthorized subjects, the images of intruders do not conform with the subspace of images of known subjects. In this context, it is the deviation from this subspace – rather than local data density – that serves as the key indicator for detecting anomalies.

Structure-based AD addresses these situations where a *prior*, expressed as a model family $\mathcal{F}$, describes genuine data and can be used to detect anomalies in a truly *unsupervised* way, *i.e.*, without any training data acting as a reference for genuine or anomalous data and without involving any training process. Structure-based AD overcomes the shortcoming of density- and distance-based techniques, which operate in the ambient data space and struggle when data lie on complex manifolds, and the shortcomings of isolation-based methods, which rely on random axis-aligned splits that ignore the implicit structure of genuine data. Structure-based AD also contrasts with the *supervised* or *semi-supervised* neural network-based AD approaches, which require a large amount of data – often unavailable or expensive to obtain – to infer the distribution of genuine data and cannot easily incorporate prior knowledge. In fact, our approach effectively leverages an explicit analytic prior, making the anomaly detection process more controllable, interpretable, and independent on data-driven training.

Figure 1a illustrates a simplified example, where the prior characterizing genuine data is the family $\mathcal{F}$ of lines, while anomalies are points deviating from instances of $\mathcal{F}$, and a density-based AD solution would fall short. In pattern recognition, rather than lines, other model families $\mathcal{F}$ describe structures or regularities in genuine data. For instance, $\mathcal{F}$ can be rototranslations in 3D registration [4] and homographies in template matching [5]. Here, anomalies, *a.k.a.* outliers, are often detected as a byproduct of a multi-model fitting process [6], where structures of inliers are first identified, and points that deviate are labeled as anomalous. Unfortunately, when multiple local structures are present, solving complex multi-model fitting problems is computationally expensive and sensitive to initialization or model assumptions. When AD is the primary goal, is preferable to employ algorithms that direclty detect anomlaies thus sidestespping the issues of multi-model fitting.

In this work, we present Preference Isolation Forest (PIF), a novel and general un-

supervised AD framework designed to identify anomalies in terms of the *preferences* they grant to genuine structures described by instances of a parametric model family $\mathcal{F}$. PIF allows for the integration of prior knowledge about genuine data, enhancing AD without requiring any training data nor trained model. PIF is composed of two main components: *i*) Preference Embedding, to map data to a high-dimensional space, called Preference Space, by fitting models from $\mathcal{F}$ (from Figure 1a to Figure 1b), and *ii*) Preference Isolation, to detect anomalies as the most *isolated* points in the Preference Space (from Figure 1b to Figure 1c). At high level, the rationale of our approach is that the preference space captures structural consistency of points, without requiring the models to be identified first and avoiding the need of ground-truth labels or supervised training. We demonstrate that in the Preference Space, when endowed with an appropriate distance (e.g., Jaccard, Tanimoto or Ruziska), anomalies naturally emerge as isolated points due to their lack of consensus with any model. We propose three main tree-based algorithms for Preference Isolation: *a*) Voronoi-iForest, that partitions the space via nested Voronoi tessellations. This is the most general one, as it can cope with any distance defined in the preference space. Then, when efficiency is a concern, we present *b*) RuzHash-iForest, that exploit RuzHash, a Locality Sensitive Hashing (LSH) [7] scheme we designed for the Preference Space to achieve a speed-up factor of $\times 35\%$ to $\times 70\%$ over Voronoi-iForest. Finally, since in real scenario the model family $\mathcal{F}$ might only approximates the genuine structures locally (see Figure 2c), we introduce *c*) Sliding-PIF a sliding window mechanism to leverage the spatial proximity in AD problems for spatial data. Sliding-PIF promotes locality, and enables scalability of AD tasks on large datasets by solving many smaller, localized AD problems, instead of a large one.

This work extends [8, 9] which introduced PIF and RuzHash-iForest respectively. In particular, regarding PIF, we provide a comprehensive overview along with a stronger and broader experimental assessment. For RuzHash-iForest, we formally prove the correctness of the RuzHash scheme, reinforcing its theoretical foundations. Moreover, the design of the Sliding-PIF algorithm is completely novel.

All in all, our contributions can be summarized as follows:

- We introduce PIF, a *general framework* for structure-based anomaly detection that leverages prior knowledge through Preference Embedding, enabling truly **unsupervised** AD without explicit model fitting.
- Building on this, we develop Voronoi-iForest, a *generic* isolation-based method for arbitrary metric spaces, suitable for preference-based representations.
- To improve efficiency, we propose RuzHash-iForest, which replaces explicit distance computations in Voronoi-iForest with a tailored LSH scheme for the Preference Space.
- Finally, we extend PIF with Sliding-PIF, incorporating data locality to enhance scalability and accuracy in settings with spatially varying structures.

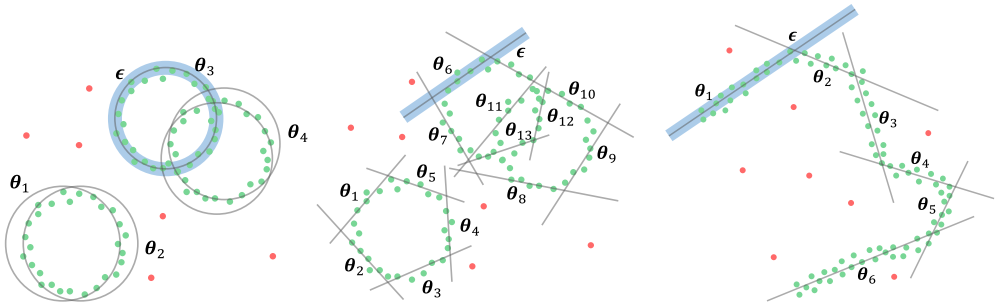


Figure 2: The same data set G can be described by different model families $\mathcal{F}$, circles in (a) and lines in (b). (c) Unknown complex patterns can be described by models that locally approximate the underlying manifold of genuine data. The inlier thresholds for some models are indicated in blue.

2. Problem Formulation

We consider a data set $X = G \cup A \subset \mathcal{X}$, where G and A are the sets of genuine and anomalous data respectively, and $\mathcal{X}$ is the ambient space. We assume that G can be described by a family $\mathcal{F}$ of known parametric models whose number is unknown, while anomalies A cannot. Specifically, each $\boldsymbol{g} \in G$ is close to the solutions of a parametric equation $\mathcal{F}(\boldsymbol{g}, \boldsymbol{\theta}) \approx 0$, that depends on unknown parameters $\boldsymbol{\theta}$. For example, in Figure 2a, $\mathcal{F}$ is the family of circles $\mathcal{F}(\boldsymbol{g}, \boldsymbol{\theta}) = (\theta_1 - g_1)^2 + (\theta_2 - g_2)^2 - \theta_3$ and, due to noise, genuine data satisfy the equation up to a tolerance, namely $|\mathcal{F}(\boldsymbol{g}, \boldsymbol{\theta})| < \epsilon$ (indicated in Figure 2 in blue), where $\epsilon > 0$ depends on the noise, whose distribution is unknown. On the other hand, anomalous data $\boldsymbol{a} \in A$ poorly satisfy the equation, *i.e.*, $|\mathcal{F}(\boldsymbol{a}, \boldsymbol{\theta})| \gg \epsilon$. Note that the same data G can be described by different model families. In Figure 2b, $\mathcal{F}$ consists of lines, expressed as $\mathcal{F}(\boldsymbol{g}, \boldsymbol{\theta}) = \theta_1 g_1 + \theta_2 g_2 + \theta_3$. The flexibility of our formulation allows $\mathcal{F}$ to *locally* describe complex structures whose underlying manifold is unknown as in Figure 2c, where G is approximated by several lines in $\mathcal{F}$ up to ϵ.

The goal of Structured-based AD is to produce an anomaly scoring function $\alpha : X \to \mathbb{R}^+$ such that $\alpha(\boldsymbol{a}) \gg \alpha(\boldsymbol{g})\ \forall \boldsymbol{a} \in A,\ \forall \boldsymbol{g} \in G$, so that anomalies are detected by setting an appropriate threshold (Figure 1c).

3. Related Work

Several AD approaches have been proposed, as extensively described in [1]. A possible taxonomy envisages four main categories: distance-based, density-based, isolation-based, and model-based including the approaches that exploit deep neural networks.

In *distance-based AD* [10] approaches, such as K-Nearest Neighbors (K-NN) [11], a point is considered anomalous when its k-nearest neighborhood is very large, thereby implicitly assuming that anomalies lie in low-density regions. Additionally, they cannot exploit prior knowledge about the family $\mathcal{F}$, and when data lie on structured manifolds (*e.g.*, lines, planes, or more complex geometric models), low-density assumption

break down. While distance-based methods are independent from data distribution and can be improved by adopting data-dependent distances [12], they are computationally intensive and perform poorly with high-dimensional data. *Density-based AD* methods, like LOF [13], identify anomalies by means of their local density, and are closely related to distance-based ones. Density-based AD methods share the computational drawbacks of distance-based ones when dealing with high-dimensional data and cannot leverage prior knowledge about family $\mathcal{F}$ as well.

Isolation-based AD methods can be traced back to Isolation Forest [14] (IFOR), where anomalies are detected as the most *"isolated"* points. IFOR builds a forest of random trees (ITREE) that recursively partition the space, and the number of splits necessary to *isolate* a point is inversely related to its anomaly score. Several extensions of IFOR were proposed: Extended Isolation Forest [15] and Generalized Isolation Forest [16] overcome the limitation of axis-parallel splits, while [17] extends IFOR to detect functional anomalies. IFOR was also extended to directly operate on pairwise point distances [18], or by different anomaly scores computation [19]. Analogously to distance and density-based ones, isolation-based methods do not take into account the underlying structure of the data, and cannot leverage priors about model family $\mathcal{F}$.

Model-based AD often treat anomaly detection as a byproduct of another task (*e.g.*, clustering), rather than as a primary objective, by first fitting a model on the data and then detecting anomalies as points that do not conform to it. When multiple or local structures are present, they typically require solving complex (multi-)model fitting problems, which are computationally expensive and sensitive to initialization or model assumptions. Examples of this approach are classification-based [20, 21], reconstruction-based [22, 23], and clustering-based methods [24]. The model-based category also encompasses deep learning methods for anomaly detection. These approaches often rely on large amounts of labeled or genuine training data, which are often unavailable or expensive to obtain. For example, recent zero-shot methods such as PointAD [25] or MVP-PClip [26], which, like our work, claim to operate without target-domain genuine data, rely on significantly different assumptions. Despite being labeled "zero-shot", they are based on extensively pre-trained Vision-Language Models like CLIP [27] to incorporate data-driven priors. These methods cannot incorporate explicit parametric structural priors (*e.g.*, geometric constraints), which are central to many real-world tasks (*e.g.*, 3D reconstruction and geometric registration). Therefore, although operating without any training example, deep learning approaches fall outside the scope of our work, which focuses on leveraging explicit structural priors, that are controllable and can be chosen to define the characteristics of genuine data.

Our proposed *structure-based AD* method PIF stands out from these approaches as it allows us to incorporate prior knowledge about the model family $\mathcal{F}$. This feature enables PIF to operate in a truly *unsupervised* manner, without requiring any training data nor trained model, thereby contrasting with the *supervised* or *semi-supervised* approach typical of classification and reconstruction-based methods. Additionally, PIF directly targets AD, in contrast to clustering-based methods that address it as a byproduct.

Algorithm 1: Preference Isolation Forest (PIF)

Input: X - data, t - number of trees, ψ - sub-sampling size, b - branching factor

Output: $\{\alpha_\psi(\boldsymbol{x}_j)\}_{j=1,\dots,n}$ - anomaly scores

`/* Preference Embedding */`

Sample m models $\{\boldsymbol{\theta}_i\}_{i=1,\dots,m}$ from X

$P \leftarrow \{\boldsymbol{p}_{\boldsymbol{x}_j} \mid \boldsymbol{p}_{\boldsymbol{x}_j} = \mathcal{E}(\boldsymbol{x}_j)\}_{j=1,\dots,n}$

`/* Voronoi-iForest construction*/`

$F \leftarrow$ Voronoi-iForest(P, t, ψ, b)

`/* Anomaly scores computation */`

for $j = 1$ to n **do**

  $D_{\boldsymbol{x}_j} \leftarrow \emptyset$

  $\boldsymbol{p}_{\boldsymbol{x}_j} \leftarrow j$-th point in P

  for $k = 1$ to t **do**

    $T_k \leftarrow k$-th Voronoi-iTree in F

    $D_{\boldsymbol{x}_j} \leftarrow D_{\boldsymbol{x}_j} \cup$ PathLength$(\boldsymbol{p}_{\boldsymbol{x}_j}, T_k, 0)$

  $\alpha_\psi(\boldsymbol{x}_j) \leftarrow 2^{-\frac{E(D_{\boldsymbol{x}_j})}{c(\psi)}}$

return $\{\alpha_\psi(\boldsymbol{x}_j)\}_{j=1,\dots,n}$

4. Method

Our Preference Isolation Forest (PIF) is a general framework for Structure-based Anomaly Detection that leverages parametric priors regarding genuine data structures in order to make unsupervised anomaly detection more effective. PIF is composed of two steps: *i*) Preference Embedding (Section 4.1), which maps points X to a high-dimensional Preference Space by means of a pool of parametric models $\{\boldsymbol{\theta}_i\}_{i=1,\dots,m}$ belonging to the family $\mathcal{F}$, and *ii*) Preference Isolation (Section 4.2), which assigns an anomaly score $\alpha(\boldsymbol{x})$ to each point $\boldsymbol{x}$ exploiting an isolation mechanism. At the core of Preference Isolation lies the construction of a forest, which can be instantiated through three variants, each designed to address different practical needs and scalability constraints. We introduce first Voronoi-iForest, a novel and general-purpose isolation-based method that serves as the backbone of our approach. Its key strength consists in supporting arbitrary distance functions between preferences through the use of Voronoi tessellations. Building upon this, we introduce RuzHash-iForest (Section 4.3), which retains the general philosophy of Voronoi-iForest but is explicitly designed to improve efficiency in large-scale settings. By leveraging LSH-based partitioning via the RuzHash mechanism, RuzHash-iForest significantly reduces computational cost and inference time, making it ideal for scenarios with massive datasets or real-time requirements. Finally, to address scenarios where genuine structures vary locally across the input space (*e.g.*, 3D surfaces), we propose Sliding-PIF (Section 4.4). This variant enhances both scalability and accuracy of PIF by operating within spatially overlapping windows, allowing adaptation to locally structured patterns that global methods

Table 1: Differences between our proposed methods Voronoi-iForest, RuzHash-iForest and Sliding-PIF.

	Purpose	Key Feature	Scenario addressed
Voronoi-iForest	Accuracy & generality	Supports arbitrary distances via Voronoi tessellations	General purpose Structure-based Anomaly Detection
RuzHash-iForest	Efficiency	Employs LSH-based partitions via RuzHash	Large-scale problems, faster inference
Sliding-PIF	Scalability & locality	Operates in spatially overlapping windows	Locally structured data (*e.g.*, 3D surfaces)

may overlook. The key features of each variants, from the generality and flexibility of Voronoi-iForest, to the efficiency of RuzHash-iForest, and ultimately to the local adaptability of Sliding-PIF are detailed in Table 1.

4.1. Preference Embedding

Preference Embedding is a function $\mathcal{E}\colon \mathcal{X} \to \mathcal{P}$ that maps data X from the ambient space $\mathcal{X}$ to a high-dimensional Preference Space $\mathcal{P} = [0,1]^m$ by leveraging prior knowledge on the parametric family $\mathcal{F}$ that well describes genuine data G. Such mapping is obtained by fitting a pool $\{\boldsymbol{\theta}_i\}_{i=1,\dots,m}$ of m models (Algorithm 1, line 1) from family $\mathcal{F}$ on data X via a RanSaC-like strategy [28]. We sample the minimal sample set – *i.e.*, the minimum number of points to constrain a model in $\mathcal{F}$ – and fit parameters $\boldsymbol{\theta}_i$. Then (line 2), we embed each sample $\boldsymbol{x} \in \mathcal{X}$ to a vector $\boldsymbol{p}_{\boldsymbol{x}} = \mathcal{E}(\boldsymbol{x}) \in \mathcal{P}$ whose i-th component is defined as:

$$p_{\boldsymbol{x},i} = \begin{cases} \phi(\delta_{\boldsymbol{x},i}) & \text{if } |\delta_{\boldsymbol{x},i}| \leq \epsilon \\ 0 & \text{otherwise} \end{cases}, \tag{1}$$

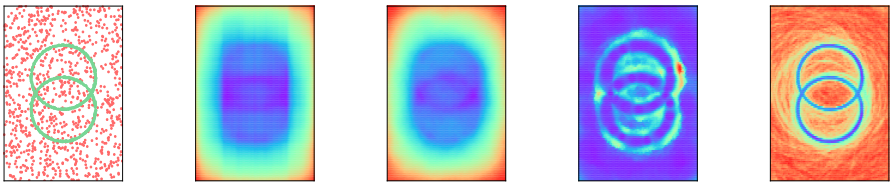
where $\delta_{\boldsymbol{x},i} = \mathcal{F}(\boldsymbol{x}, \boldsymbol{\theta}_i)$ is the residual of $\boldsymbol{x}$ with respect to $\boldsymbol{\theta}_i$, and $\epsilon = k\sigma$ is an inlier threshold proportional to the noise standard deviation $\sigma > 0$. We assume the amount of noise to be unknown, but directly estimable from data (*e.g.*, [29]). As a consequence, setting the inlier threshold can often be treated as an educated guess, guided by established statistical heuristics. A common practice is to choose k such that a desired percentage of inliers fall below the threshold—following, for instance, the three-sigma rule, which retains approximately 99.7% of data under the assumption of Gaussian noise. This principle can be also adapted to other noise distributions. The preference function ϕ is defined as:

$$\phi(\delta_{\boldsymbol{x},i}) = e^{-\frac{\delta^2_{\boldsymbol{x},i}}{2\sigma^2}}.$$

We considered also a *binary* preference function, that is $\phi(\delta_{\boldsymbol{x},i}) = 1$ when $|\delta_{\boldsymbol{x},i}| \leq \epsilon$ and 0 otherwise, giving rise to a discrete preference space $\{0,1\}^m$. Unless otherwise specified, we refer to $\mathcal{P} = [0,1]^m$ as the *continuous* Preference Space. As illustrated in Figure 1, a family $\mathcal{F}$ describing genuine data can isolate anomalies in $\mathcal{P}$, making them detectable via Preference Isolation.

4.1.1. Choice of model family $\mathcal{F}$

The model family $\mathcal{F}$ in (1) encodes the prior knowledge on genuine data G in the Preference Space, which can be either *global* or *local*. In Figure 2a, where G refers to circles, choosing $\mathcal{F}$ as the family of circles provides a *global* description of genuine



(a) Input data. (b) iFor [14] (c) EiFor [15] (d) LOF [13] (e) PIF

Figure 3: Anomaly scores $\alpha(\cdot)$, color coded, computed by different algorithms. PIF effectively detects anomalies (red points in (a)) when a suitable family $\mathcal{F}$ (circles) is chosen for Preference Embedding.

data. In Figure 2c we show that, even if the family of G is unknown, it is possible to adopt lines to describe G locally. This results in *local* models $\boldsymbol{\theta}_i$ that describe accurately the manifold only in a neighborhood of $g \in G$, but in Section 4.4 we will show that this is enough to detect anomalous data.

Remark. In principle, if the genuine data were generated by a single underlying model, one could first estimate this model robustly, and then detect anomalies as points with the highest deviation from the model. However, in our scenario, the data might arise from a mixture of unknown multiple structures, which may be valid only locally. Even if all genuine models were available, it would not be trivial to determine which residual $\delta_{x,i}$ (with respect to model θ_i) should be considered for a given point x. Rather than solving the difficult and ill-posed problem of multi-model fitting, and instead of relying on residuals directly, we embed points into the preference space, which captures how consistently each point agrees with the model set randomly sampled. This sidesteps model estimation entirely, and enables robust anomaly detection through isolation in preference space, where outliers naturally emerge as structurally inconsistent points.

4.2. Preference Isolation

Preference Isolation consists in a scheme to isolate anomalies in the Preference Space $\mathcal{P}$, resulting in better detection with respect to methods that operate in the ambient space $\mathcal{X}$. This is illustrated in Figure 3, where traditional isolation-based methods operating in $\mathcal{X}$ struggle to detect anomalies inside the circle, while PIF succeeds in this challenging task.

4.2.1. Distances in Preference Space

The effectiveness of PIF is closely related to the ad-hoc distance functions adopted to isolate anomalies in the Preference Space. Several works in the literature [30, 31, 8, 9] investigates distances in the Preference Space:

Jaccard. The Jaccard [32] distance is an indicator of dissimilarity between two sets. Given $\boldsymbol{p}, \boldsymbol{q}$ belonging to the binary Preference Space $\{0,1\}^m$ [30], their Jaccard distance is defined as:

$$d_J(\boldsymbol{p}, \boldsymbol{q}) = 1 - \frac{\sum_{i=1}^{m}(p_i \wedge q_i)}{\sum_{i=1}^{m}(p_i \vee q_i)}, \tag{2}$$

where $\wedge$ and $\vee$ denote the logical *intersection* and *union*, respectively.

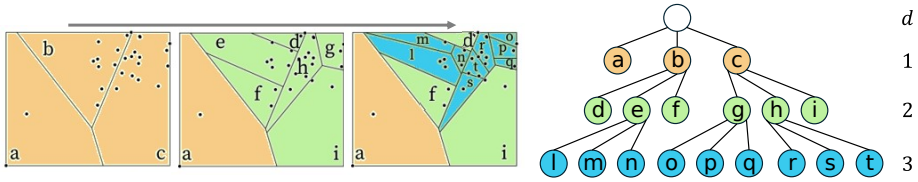


Figure 4: Voronoi-iTree with branching factor $b = 3$ and depth limit $l = 3$ built on points in $\mathbb{R}^2$. Regions are recursively split in b sub-regions. The most isolated samples fall in leaves at lowest depth, such as `a` and `i`.

Algorithm 2: Voronoi-iTree

Input: P - preferences, e - tree depth, l - depth limit, b - branching factor

Output: T - a Voronoi-iTree

if $e \geq l$ or $|P| < b$ **then**
  return $exNode\{Size \leftarrow |P|\}$
else
  $\{s_i\}_{i=1,\dots,b} \leftarrow \textsc{Subsample}(P, b)$
  $\{P_i\}_{i=1,\dots,b} \leftarrow \textsc{voronoiPartition}(P, \{s_i\}_{i=1,\dots,b})$
  $chNodes \leftarrow \emptyset$
  for $i = 1$ to b **do**
    $chNodes \leftarrow chNodes \cup$ Voronoi-iTree$(P_i, e + 1, l, b)$
  return $inNode\{ChildNodes \leftarrow chNodes, SplitPoints \leftarrow \{s_i\}_{i=1,\dots,b}\}$

Ruzicka. The Ruzicka [33] distance extends Jaccard distance to vectors with values in $[0, 1]^m$. Analogously to (2), the Ruzicka distance is defined as:

$$d_R(\boldsymbol{p}, \boldsymbol{q}) = 1 - \frac{\sum_{i=1}^{m} \min\{p_i, q_i\}}{\sum_{i=1}^{m} \max\{p_i, q_i\}}.$$

Tanimoto. The Tanimoto [34] distance extends Jaccard as follows:

$$d_T(\boldsymbol{p}, \boldsymbol{q}) = 1 - \frac{\langle \boldsymbol{p}, \boldsymbol{q} \rangle}{\|\boldsymbol{p}\|^2 + \|\boldsymbol{q}\|^2 - \langle \boldsymbol{p}, \boldsymbol{q} \rangle} = 1 - \frac{\sum_{i=1}^{m} p_i q_i}{\sum_{i=1}^{m} (p_i^2 + q_i^2) - \sum_{i=1}^{m} p_i q_i}.$$

It is worth noting that these three distances reach their maximum when $\boldsymbol{p}, \boldsymbol{q}$ are orthogonal ($\boldsymbol{p} \perp \boldsymbol{q}$). Since anomalies become sparse vectors in $\mathcal{P}$, as they are typically poorly described by fitted models, their preference vectors are nearly orthogonal to those of genuine data, becoming isolated points in $\mathcal{P}$. Figure 1b shows the MDS of the points in the Preference Space with reference to Tanimoto distance. Genuine data G from line $\boldsymbol{\theta}_1$ are close in the Preference Space, but distant to those from line $\boldsymbol{\theta}_2$, while anomalies A are both distant to each other and to genuine data G, resulting *isolated*.

Algorithm 3: Voronoi-iForest

Input: P - preference representations, t - number of trees, ψ - sub-sampling size, b - branching factor

Output: $F = \{T_k\}_{k=1,\dots,t}$ - set of Voronoi-iTrees

$F \leftarrow \emptyset$
$l \leftarrow ceiling(\log_b \psi)$
for $k = 1$ to t **do**
    $P_\psi \leftarrow$ Subsample(P, ψ)
    $T_k \leftarrow$ Voronoi-iTree$(P_\psi, 0, l, b)$
    $F \leftarrow F \cup T_k$
return F

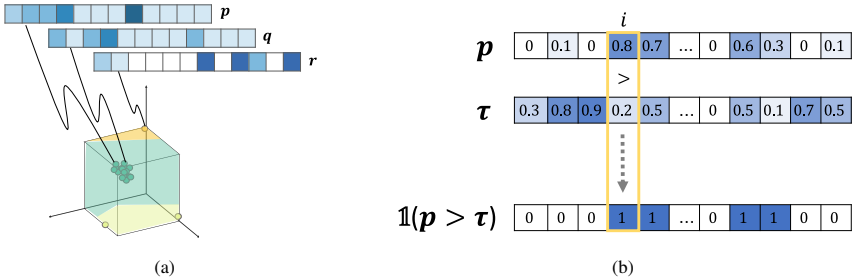
*4.2.2. Voronoi-*iTree

Voronoi-iTree is the starting point of our framework for Preference Isolation. It is designed to be the most *general* algorithm, as it can seamlessly adapt to any distance measure defined in the preference space – such as those discussed in the previous section. In fact, Voronoi-iTree relies on Voronoi tessellations, and can be extended to operate directly in the ambient space when appropriate. As such, it serves as a consistent reference for evaluating detection accuracy varying the distances adopted, and illustrates the benefits of Preference Isolation within Preference Space.

We employ the previously defined distances within a tree-based recursive partitioning algorithm, where points are grouped in each level based on their similarity according to the selected distance, to identify anomalies as the most isolated points in the Preference Space $\mathcal{P}$. Our approach, Voronoi-iTree, consists in a nested configuration of Voronoi tessellations, where each cell is recursively split into b sub-regions, as illustrated in Figure 4 where $b = 3$. The construction of a Voronoi-iTree, described in Algorithm 2, starts by randomly sampling b Voronoi seeds $\{\boldsymbol{s}_i\}_{i=1,\dots,b} \subset P$ (Algorithm 2) to partition the set of preferences P (Algorithm 2). Specifically, P is split into b subsets $\{P_i\}_{i=1,\dots,b}$, where each $P_i \subset P$ collects all the points having $\boldsymbol{s}_i$ as the closest seed according to the selected distance. Each P_i is then used to build sub-trees recursively (Algorithm 2). The recursive partitioning process stops when the number of points in the region P_i is less than b, or the tree reaches a maximum depth (Algorithm 2) set to $l = \log_b \psi$ as in [35], where ψ is the number of points used to build the tree. To decrease the randomness in Voronoi-iTree construction, we build an ensemble of t Voronoi-iTrees, termed Voronoi-iForest (Algorithm 3), where each Voronoi-iTree is built on a random subset $P_\psi \subset P$.

4.2.3. Anomaly score computation

We compute the anomaly score of a point $\boldsymbol{x} \in \mathcal{X}$ as a function of its *average* depth within the Voronoi-iForest. The rationale is that *isolated* points fall in leaves at lower depth as in [14]. We then collect the depths reached by each preference $\boldsymbol{p}_{\boldsymbol{x}} = \mathcal{E}(\boldsymbol{x}) \in P$, in each of the t trees of the forest, into a set $D_{\boldsymbol{x}}$ (Algorithm 1, Algorithm 1). Then



(a) (b)

Figure 5: (a) RuzHash induces a partitioning of the preference space $\mathcal{P}$ (color-coded here), where close points with respect to the Ruzicka distance $(\boldsymbol{p}, \boldsymbol{q})$ are more likely mapped to the same region, unlike isolate points $(\boldsymbol{r})$. We use RuzHash as the splitting criterion within each node of RuzHash-iTree. (b) An example of binarization function used in Item 3.

(Algorithm 1), we compute the anomaly score as:

$$\alpha_{\psi}(\boldsymbol{x}) = 2^{-\frac{E(D_{\boldsymbol{x}})}{c(\psi)}}, \tag{3}$$

where $E(D_{\boldsymbol{x}})$ is the mean value over the elements of $D_{\boldsymbol{x}}$, and $c(\psi)$ is an adjustment factor. The depth of each $\boldsymbol{p_x}$ is computed via the PathLength function, following a recursive tree traversal to the leaf containing $\boldsymbol{p_x}$, similar to [14]. As in [14], the depth is computed as $d(\boldsymbol{p_x}) = e + c(exNode.Size)$, where e represents the leaf node depth, and $c(\cdot)$ is an adjustment factor to account for samples that fell in the leaf during the tree construction process, but did not contribute to the depth due to the depth limit l (Algorithm 2, Algorithm 2).

4.3. Dealing with efficiency: hashing to speed up distance computation

Voronoi tessellations are effective in isolating preference vectors corresponding to anomalies, but they are inefficient when the number of points n is large, as they require to compute distances between each of the n points $\boldsymbol{p_x} \in P$ and seeds $\{\boldsymbol{s}_i\}_{i=1,\dots,b}$, leading to a computational complexity of $O(n\,b)$ for each Voronoi tessellation. To overcome this bottleneck, we revisit the way in which the distances between preferences are computed and introduce RuzHash, a novel Locality Sensitive Hashing (LSH) scheme that splits points in the Preference Space with complexity $O(n)$, and ensures that nearby points are more likely mapped to the same bin without computing point-to-point distances. At high level, the bins obtained via LSH, play the same role of the Voronoi cells used in the Voronoi-iTree.

4.3.1. RuzHash

RuzHash is a mapping $h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz} : [0,1]^m \rightarrow \{1,\dots,m\}$ from the Preference Space to a collection of m bins. The main idea, depicted in Figure 5a, is that when two points $\boldsymbol{p}, \boldsymbol{q} \in \mathcal{P}$ are close with respect to Ruzicka distance, they are likely mapped to the same bin with high probability, *i.e.*, $d_R(\boldsymbol{p}, \boldsymbol{q}) \approx 0 \iff Pr[h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{q})] \approx 1$, while a point $\boldsymbol{r}$ far from $\boldsymbol{p}$ is likely mapped to a different bin, *i.e.*, $d_R(\boldsymbol{p}, \boldsymbol{r}) \approx 1 \iff$

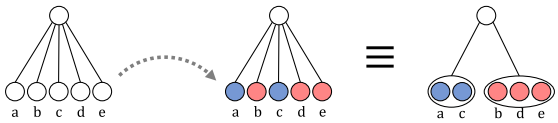


Figure 6: Left: split performed by RuzHash when $m = 5$. Middle: nodes aggregation where the groups are color coded. Right: resulting tree with branching factor $b = 2$.

$Pr[h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{r})] \approx 0$. In practice, given a point $\boldsymbol{p} \in [0,1]^m$, the RuzHash mapping is computed as follows:

1. sample a vector of thresholds $\boldsymbol{\tau} = [\tau_1, \ldots, \tau_m]$, where $\tau_i \sim \mathcal{U}_{[0,1)}$,
2. sample a random permutation $\boldsymbol{\pi} = [\pi_1, \ldots, \pi_m]$ of the indices $\{1, \ldots, m\}$,
3. map $\boldsymbol{p}$ to the bin having index $h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{p}) = \min(\boldsymbol{\pi} \cdot \mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau}))$, where $\mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau}) = [\mathbb{1}(p_1 > \tau_1), \ldots, \mathbb{1}(p_m > \tau_m)] \in \{0,1\}^m$ is a binarization function (Figure 5b), and the minimum is computed over the indices.

The binarization function uses the thresholds vector $\boldsymbol{\tau}$ to map points from the continuous Preference Space $[0,1]^m$ into the binary Preference Space $\{0,1\}^m$, while the permutation $\boldsymbol{\pi}$ and $\min(\cdot)$ operator are used as in MinHash [36], a Locality Sensitive Hashing (LSH) scheme for the Jaccard distance on binary vectors. We prove that the RuzHash mapping is an LSH of the Ruziska distance, ensuring that similar points are more likely to be mapped to the same bin, as stated in the following

Theorem 1. *Given $\boldsymbol{p}, \boldsymbol{q} \in [0,1]^m$, then:*

$$Pr[h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{q})] = 1 - d_R(\boldsymbol{p}, \boldsymbol{q}).$$

The proof is in Appendix A, where Figure A.11 also shows the empirical correlation between Ruzicka distance and its approximation via RuzHash.

4.3.2. RuzHash-iTree

At each step of the recursive tree building process, RuzHash splits the set of preferences P into exactly m bins, where m is the dimension of Preference Space $\mathcal{P} = [0,1]^m$, as illustrated in Figure 6 (left). Since m is often larger than the number of points n, most of the bins remain empty, causing unnecessary memory usage. Additionally, a high branching factor m results in few tree levels, leading to less fine grained anomaly scores. To address this, we reduce the branching factor $b < m$ by randomly aggregating the m bins produced by RuzHash, as shown in Figure 6 (right). We perform this aggregation in two steps: *i*) we sample a vector $\boldsymbol{\beta} = [\beta_1, \ldots, \beta_m]$, where $\beta_i \sim \mathcal{U}_{\{1,\ldots,b\}}$, to assign a random value from $\{1, \ldots, b\}$ to each bin produced by RuzHash (represented by different colors in Figure 6, middle), and *ii*) we define a new RuzHash mapping $h^{\boldsymbol{\pi},\boldsymbol{\tau},\boldsymbol{\beta}}_{ruz} : [0,1]^m \to \{1, \ldots, b\}$ such that $h^{\boldsymbol{\pi},\boldsymbol{\tau},\boldsymbol{\beta}}_{ruz}(\boldsymbol{p}) = \beta_{h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}}$. This aggregation remaps each bin $\{1, \ldots, m\}$ produced by RuzHash to a new bin $\{1, \ldots, b\}$, reducing the branching factor from m to b.

Algorithm 4: RuzHash-iTree

Input: P - preferences, e - tree depth, l - depth limit, b - branching factor

Output: T - a RuzHash-iTree

if $e \geq l$ or $|P| < b$ **then**
  return $exNode\{Size \leftarrow |P|\}$
else
  $\boldsymbol{\tau}, \boldsymbol{\pi} \leftarrow$ RuzHashSample(m)
  $\boldsymbol{\beta} \leftarrow$ AggregationSample(m, b)
  $\{P_i\}_{i=1,\dots,b} \leftarrow$ RuzHashPartition($P, \boldsymbol{\tau}, \boldsymbol{\pi}, \boldsymbol{\beta}$)
  $chNodes \leftarrow \emptyset$
  for $i = 1$ to b **do**
    $chNodes \leftarrow chNodes \cup$ RuzHash-iTree($P_i, e+1, l, b$)
  return $inNode\{ChildNodes \leftarrow chNodes, Thresholds \leftarrow \boldsymbol{\tau}$
    $Permutation \leftarrow \boldsymbol{\pi}, Aggregation \leftarrow \boldsymbol{\beta}\}$

Table 2: Differences between iForest, Voronoi-iForest and RuzHash-iForest.

	Splitting scheme	Distance	Computational complexity	
			Training	Testing
Voronoi-iForest	Voronoi	Tanimoto	$O(\psi\, t\, b\, \log_b \psi)$	$O(n\, t\, b\, \log_b \psi)$
iForest	LSH	ℓ_1	$O(\psi\, t\, \log_2 \psi)$	$O(n\, t\, \log_2 \psi)$
RuzHash-iForest	LSH	Ruzicka	$O(\psi\, t\, \log_b \psi)$	$O(n\, t\, \log_b \psi)$

Algorithm 4 details the construction of RuzHash-iTree, where in Algorithm 4 we sample the random vectors, and in Algorithm 4 we split P via RuzHash. In Appendix B, we theoretically study the impact of the branching factor b on RuzHash, showing that it influences the correlation slope between RuzHash and Ruzicka. Table 2 summarizes key differences between iForest, Voronoi-iForest and RuzHash-iForest.

4.4. Dealing with locality: a sliding window approach

Here we present Sliding-PIF, an efficient variation of PIF that exploits a *locality principle*: points close in the ambient space $\mathcal{X}$ are likely to belong to the same genuine structure. This turns to be useful when the family $\mathcal{F}$ approximates the genuine data only locally (see Figure 2c), and anomalies deviate from the regularity of smooth genuine structures.

4.4.1. Sliding-PIF

In many real-world problems, particularly in geometric settings—a global structure is either absent or too complex to capture with a single model, but local geometric regularities remain clearly identifiable. A representative example is that of 3D surface scans, where small neighborhoods can be approximated by simple geometric primitives (e.g., planes), even though the full surface may exhibit significant curvatures. In these scenarios, adopting a global preference embedding may be impractical or inefficient, as it would require an excessive number of model instances to capture the diverse



Figure 7: (a) Preferences are meaningful within the approximated region. (b) Range images connectivity enables efficient neighborhood definition.

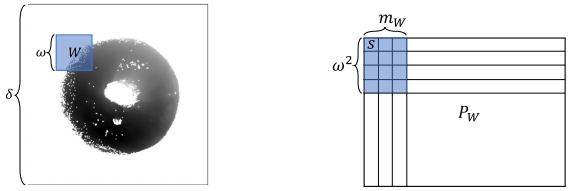
local phenomena, resulting in increased computational and memory demands. Sliding-PIF addresses this challenge by applying PIF locally, within spatially overlapping regions, allowing adaptation to spatial variations in structure. This approach preserves the strengths of the PIF framework while improving both scalability and accuracy, and illustrates its modular and flexible nature when applied to complex geometric data.

Specifically, Sliding-PIF leverages the *locality* principle by fitting models on points sampled from windows W and performing structure-based AD independently within each window. To present Sliding-PIF, we consider range images, which can be represented as 3D surfaces as illustrated in Figure 7b, and where we can efficiently exploit the locality principle since data are structured in a grid format. We assume that genuine data G lie on a smooth surface $\mathcal{S}$, thus a genuine point $\boldsymbol{g}$ can be locally described by its tangent plane $\boldsymbol{\theta}$, while anomalies A (defects, such as cracks) cannot. Rather than considering the whole dataset, we operate in a window-wise fashion by first selecting overlapping windows, and then performing PIF on the corresponding set of points X_W, producing an anomaly score $\alpha_\psi^W(\boldsymbol{x})$ (3) for each point $\boldsymbol{x} \in X_W$ and for each window W it belongs to. The window-wise approach has two advantages: *i)* we draw minimal sample sets from nearby points, promoting the extraction of genuine local models as in localized sampling strategies [37], and *ii)* we compute preferences only for points within the local region of interest, yielding significant computational gain (Figure 7a). Finally, we compute the overall anomaly score for each point $\boldsymbol{x}$ by averaging scores $\alpha_\psi^W(\boldsymbol{x})$ across all windows.

The choice of the window size ω is closely tied to the size of the anomalous region, and it must be large enough to ensure anomalies remain a minority within the window. We found that windows overlapping by half of their width strike a good balance between efficiency and effectiveness.

Sliding-PIF benefits over PIF in terms of memory usage. Figure 8a shows an input image of size $\delta \times \delta$, a window W of size $\omega = \frac{\delta}{k}$, and Figure 8b shows the Preference Embedding P_W corresponding to W. The size of P_W is proportional to the $\omega^2 = \frac{\delta^2}{k^2}$ points in W and to the m_W models fitted. Thus, the memory needed is $s\frac{\delta^2}{k^2}m_W$, where s is the memory size of a preference value p. The total number of half-overlapping windows W of size $\omega = \frac{\delta}{k}$ is $(2\frac{\delta}{\omega} - 1)^2 = (2k-1)^2$, resulting in a total memory need of

$$P_{size} = s\frac{d^2}{k^2}m_W(2k-1)^2.$$



(a) (b)

Figure 8: (a) Bagel image with side length δ and superimposed window W of size ω, and (b) the corresponding local Preference Embedding matrix P_W.

Thus, if we fix the total memory size to P_{size}, we can fit

$$m_W = \left\lfloor \frac{P_{size}}{s\frac{d^2}{k^2}(2k-1)^2} \right\rfloor$$

models. As instance, considering $s = 32$ bytes, $\delta = 800$ and $P_{size} = 1$ GB, *each window* W can accommodate $m_W = 419$ models for $\omega = \delta$ and $m_W = 110$ models for $\omega = \frac{\delta}{20}$, where PIF is Sliding-PIF with window size $\omega = \delta$, resulting in a total number of models m of 419 and 167310 respectively.

5. Experiments

In this section, we evaluate the benefits of PIF on synthetic and real datasets, showing its superior performance over state-of-the-art AD methods, and we demonstrate both the importance of leveraging our structure-based prior on genuine data and the need of employing the suitable distances in the Preference Space (Section 5.2). We then evaluate the efficiency gain of RuzHash-iForest over Voronoi-iForest (Section 5.3) and assess the benefits of Sliding-PIF on data satisfying the locality principle (Section 5.4).

5.1. Datasets

As regard synthetic datasets, we consider the 2D primitive fitting [8, 9] comprising points $X \in \mathbb{R}^2$ where genuine data G lie on lines or circles, and anomalies are uniformly sampled within the range of G with $\frac{|A|}{|X|} = 0.5$. As far as real data are concerned, we consider AdelaideRMF dataset, comprising stereo images with annotated matching points, where anomalies are mismatches. The first 19 images are static scenes with many planes, giving rise to genuine matches described by homographies. The last 19 images are dynamic scenes with moving objects, genuine matches here are described by fundamental matrices.

To evaluate the impact of local priors, we also considered the MVTec-3D AD [2] dataset, containing range images of 10 different objects with various defects, such as holes and cracks. Scans were acquired using structured light, with positions x, y, and z stored in three-channel tensors. We discard the color information and focus solely

Table 3: Optimal k parameters of LOF

	Euclidean	Preference binary	Preference
circle	75	25	25
line	25	150	75
homography	–	100	100
fundamental	–	75	80

on the geometric data. We define and evaluate two distinct scenarios. i) *Smooth objects scenario*: this includes objects that are locally smooth and thus conform well to the locality and smoothness assumptions underlying Sliding-PIF. Specifically, we select *cookie*, *peach*, *potato*, and *carrot*. For these objects, the genuine structures are approximately smooth in local neighborhoods, and anomalies such as holes manifest as localized geometric irregularities. We also restrict our evaluation to the hole defect type, which directly violates the local smoothness assumption. ii) *Mechanical objects scenario*: to assess the limits of our method, we also consider a set of mechanical objects, namely *cable gland*, *dowel*, *foam*, *rope*, and *tire*, which are characterized by sharp edges, corners, and discontinuities. In these cases, the assumption of local smoothness is frequently violated even in non-defective regions, making the detection problem more challenging and testing the robustness of our approach in settings that deviate from its ideal operating assumptions.

5.2. The effectiveness of structure-based anomaly detection

Regarding synthetic datasets, we evaluated performance across multiple structured and controlled scenarios to assess the utility of the preference embedding. We compare PIF to iFor [14], EiFor [15] and LOF [13] either in the ambient space $\mathbb{R}^2$, the binary Preference Space $\{0,1\}^d$ and the continuous Preference Space $[0,1]^d$. We compute preferences with respect to $m = 10|X|$ model instances, and we tested Preference Isolation respectively with *i*) Voronoi-iForest Euclidean (ViFor ℓ_2) in the ambient space $\mathbb{R}^2$, *ii*) Voronoi-iForest Jaccard (ViFor jac) in the binary Preference Space $\{0,1\}^d$, and *iii*) Voronoi-iForest Tanimoto (ViFor tani), Voronoi-iForest Ruzicka (ViFor ruz) and RuzHash-iForest (RzHiFor) in the continuous Preference Space $[0,1]^d$.

In this synthetic scenario, we also considered as a baseline the Euclidean distances computed directly in the ambient space, which is meaningful in this setting due to the low dimensionality of the data ($\mathbb{R}^2$) and its geometric interpretability. Moreover, inliers tend to be spatially denser and more structured, whereas outliers are often uniformly distributed over the plane. In contrast, for real-world datasets the situation is significantly different: the ambient space is higher-dimensional, as data represent point correspondences between two images, which naturally reside in the product of two projective planes and can be embedded in $\mathbb{R}^6$. In such a high-dimensional and inherently non-Euclidean space, standard Euclidean distances become much less informative for detecting anomalies. For this reason, we perform detection exclusively in the continuous Preference Space, where geometric priors (*e.g.*, homographies or fundamental matrices) can be meaningfully exploited. In this setup we compute preferences on $m = 6|X|$ models.

Parameters of iFor, EiFor, ViFor and RzHiFor are fixed to $t = 100, \psi = 256$ and

Table 4: Synthetic datasets AUCs

	Euclidean				Preference binary				Preference					
	LOF ℓ_2	iFor	EiFor	ViFor ℓ_2	LOF jac	iFor	EiFor	ViFor jac	LOF tani	iFor	EiFor	ViFor tani	ViFor ruz	RzHiFor
stair3	0.737	0.925	0.920	0.918	0.904	0.885	0.864	0.958	0.815	0.923	0.925	0.971	0.969	0.959
stair4	0.814	0.889	0.874	0.871	0.849	0.855	0.860	0.941	0.881	0.912	0.908	0.952	0.950	**0.965**
star5	0.771	0.722	0.738	0.788	0.875	0.745	0.769	0.872	**0.929**	0.761	0.822	0.910	0.869	0.841
star11	0.671	0.728	0.727	0.738	0.830	0.739	0.741	0.771	**0.900**	0.738	0.774	0.796	0.817	0.714
circle3	0.761	0.698	0.732	0.779	0.719	0.842	0.854	0.900	0.731	0.854	0.891	**0.930**	0.851	0.871
circle4	0.640	0.641	0.665	0.679	0.827	0.686	0.699	0.860	0.906	0.667	0.720	0.897	0.777	0.722
circle5	0.543	0.569	0.570	0.633	0.699	0.597	0.617	0.672	**0.823**	0.573	0.593	0.780	0.565	0.638
Mean	0.705	0.739	0.747	0.772	0.815	0.764	0.772	0.853	0.855	0.775	0.805	**0.891**	0.828	0.816

Table 5: Real datasets AUCs

(a) Homographies

	LOF tani	iFor	EiFor	ViFor tani	ViFor ruz	RzHiFor
barrsmith	**0.969**	0.708	0.715	0.944	0.698	0.692
bonhall	0.918	0.969	0.967	0.949	0.960	0.951
bonython	**0.978**	0.679	0.691	0.954	0.911	0.857
elderhalla	0.999	0.925	0.909	**0.999**	0.878	0.877
elderhallb	0.986	0.924	0.943	**0.999**	0.966	0.976
hartley	0.963	0.749	0.793	**0.989**	0.911	0.882
johnsona	0.993	0.993	0.993	**0.998**	0.962	0.993
johnsonb	0.776	0.999	0.998	0.999	0.929	0.980
ladysymon	0.847	0.944	0.943	**0.997**	0.942	0.983
library	**1.000**	0.764	0.771	0.998	0.968	0.936
napiera	0.975	0.869	0.879	**0.983**	0.756	0.791
napierb	0.888	0.931	0.936	**0.953**	0.938	0.948
neem	0.985	0.896	0.906	**0.996**	0.928	0.969
nese	**0.996**	0.888	0.892	0.980	0.920	0.958
oldclassicswing	0.936	0.923	0.943	0.987	**0.998**	0.993
physics	0.670	0.858	0.787	**1.000**	0.989	0.969
sene	**0.997**	0.698	0.731	0.988	0.936	0.763
unihouse	0.785	0.998	0.998	**0.999**	0.930	0.980
unionhouse	**0.987**	0.639	0.664	0.968	0.877	0.840
Mean	0.929	0.861	0.866	**0.983**	0.916	0.913

(b) Fundamental matrices

	LOF tani	iFor	EiFor	ViFor tani	ViFor ruz	RzHiFor
biscuit	0.976	0.994	0.996	**1.000**	0.922	0.967
biscuitbook	1.000	0.987	0.988	1.000	0.945	0.986
biscuitbookbox	**1.000**	0.990	0.989	0.996	0.895	0.982
boardgame	**0.962**	0.400	0.304	0.949	0.750	0.883
book	0.996	1.000	1.000	1.000	0.925	0.982
breadcartoychips	**0.989**	0.978	0.971	0.976	0.827	0.975
breadcube	**1.000**	0.998	0.998	0.999	0.777	0.979
breadcubechips	**0.999**	0.985	0.985	0.998	0.861	0.976
breadtoy	0.984	0.999	0.998	0.999	0.920	0.984
breadtoycar	**0.998**	0.933	0.883	0.991	0.827	0.957
carchipscube	**0.993**	0.981	0.966	0.987	0.829	0.975
cubetoy	**1.000**	0.997	0.995	1.000	0.960	0.987
cube	0.999	0.970	0.982	0.999	0.895	0.952
cubebreadtoychips	0.990	0.962	0.958	0.989	0.752	0.930
cubechips	**1.000**	0.995	0.994	1.000	0.980	0.996
dinobooks	0.887	0.873	0.857	0.899	0.879	**0.946**
game	**1.000**	0.901	0.895	0.999	0.844	0.898
gamebiscuit	1.000	0.985	0.988	**1.000**	0.876	0.981
toycubecar	**0.973**	0.290	0.192	0.964	0.742	0.935
Mean	0.987	0.906	0.891	0.987	0.864	0.962

$b = 2$ in all experiments following standard practice iFor [14]. Regarding LOF, we vary k, ranging from $k = 10$ to $k = 500$, and keep the best result.

The AD performance is evaluated with AUC averaged over 10 runs (Tables 4, 5a and 5b). The highest value for each dataset is underlined, and boldface indicates statistically better methods, according to paired t-test ($\alpha = 0.05$). LOF results refer to the k with highest AUC along datasets of each family $\mathcal{F}$, for all the embeddings (k values reported in Table 3).

Table 4 shows that all methods improve their performance when run in Preference Space, thus confirming the benefits of using the prior $\mathcal{F}$ rather than working with density in the ambient space, with continuous preferences to be preferred. LOF performs well on *star5*, *star11*, *circle4* and *circle5*, where genuine data points are evenly distributed among structures, *i.e.*, the different structures have similar cardinality. In such balanced settings, it is easier to choose an appropriate neighborhood size k, as each inlier has a sufficient number of nearby points belonging to the same structure, reducing the risk of misclassification due to uneven density. By contrast, LOF performs poorly when structures have different cardinalities sizes (*stair3* and *circle3*), while ViFor and RzHiFor (run with fixed parameters) achieve superior performance, with ViFor the

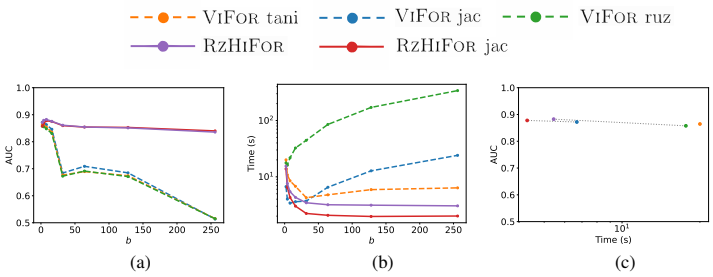


Figure 9: Results on synthetic and real datasets, averaged together, at various branching factors. (a) Average AUCs. (b) Average test times. (c) Best average AUCs and corresponding test time.

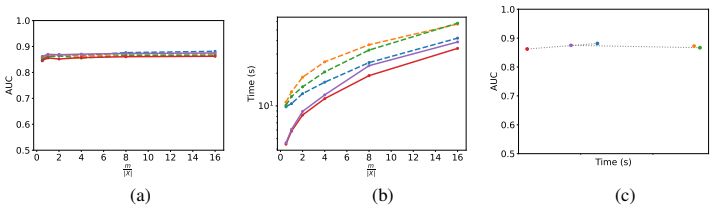


Figure 10: Results on synthetic and real datasets, averaged together, at various number of models. (a) Average AUCs. (b) Average test times. (c) Best average AUCs and corresponding test time.

best statistically significant method. ViFor and RzHiFor outperform iFor and EiFor as they leverage the most suitable distances for Preference Space. On real data, high AUC values in Table 5a and Table 5b show that AD is easier than the synthetic case, still the performance improvement ViFor, RzHiFor and LOF over iFor and EiFor remains evident.

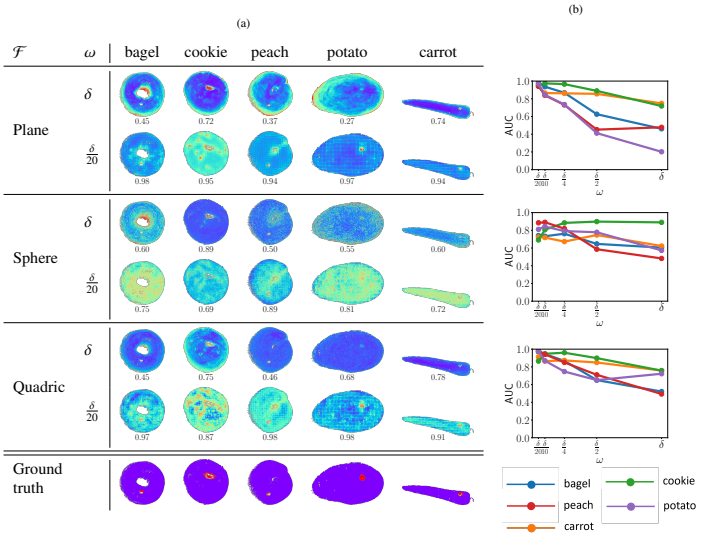
5.3. The impact of hashing on efficiency

We perform an extensive comparison between RuzHash-iForest and Voronoi-iForest both in the binary and continuous Preference Space on the same datasets used for Section 5.2. We tested Voronoi-iForest and RuzHash-iForest with same parameters: $t = 100$, $\psi = 256$, but varying branching factors in $b \in \{2, 4, 8, 16, 32, 64, 128, 256\}$ and number of models $m \in \{0.5|X|, |X|, 2|X|, 4|X|, 8|X|, 16|X|\}$.

Figure 9a and Figure 9b show the average AUC and test time curves at various b values, produced by averaging the results of 5 executions on synthetic and real datasets separately, then averaging them together. RuzHash-iForest shows the most stable AUC with respect to b, and an evident efficiency gain consistent with the complexities in Table 2. Figure 9c relates the best AUC and corresponding test time of methods referring to the same distance function. RuzHash-iForest is as accurate as Voronoi-iForest, but with gain in time. RzHiFor jac is $\times 35\%$ faster than ViFor jac, and RzHiFor is $\times 70\%$ faster than ViFor ruz.

Similarly, Figure 10a and Figure 10b show the average AUC and test time curves for various values of m. All methods exhibit stable performance across different m

Table 6: (a) Qualitative results of PIF ($\omega = \delta$) and Sliding-PIF ($\omega = \frac{\delta}{20}$), alongside ground truths, when executed on smooth objects. Anomaly scores $\alpha(\cdot)$ color-coded (red indicates anomalies), and AUC values displayed below objects. (b) Average AUC values on 5 runs achieved by PIF and Sliding-PIF at different ω.



values, indicating that the sensitivity to this parameter is less critical. Increasing m generally leads to longer execution times without yielding significant improvements in detection performance.

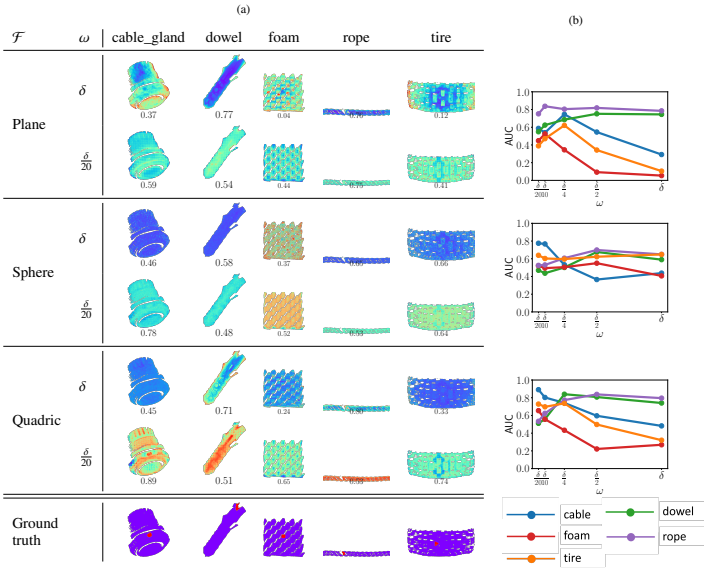
5.4. The impact of sliding window

We assess Sliding-PIF on a real-world scenario, namely the MVTec-3D AD dataset, using families of local models $\mathcal{F}$ with increasing generality (planes, spheres, and quadrics). Notably, we do not rely on any ground-truth genuine or anomalous data, since Sliding-PIF is a truly unsupervised method. We ran Sliding-PIF with different window sizes ω to account for various defect sizes ($\omega = [\delta, \frac{\delta}{2}, \frac{\delta}{4}, \frac{\delta}{10}, \frac{\delta}{20}]$, where δ is the image side length). For Preference Isolation we used RuzHash-iForest in the continuous Preference Space with $t = 100$, $\psi = 256$ and $b = 16$ for greater efficiency.

We evaluate the impact of the window size ω on qualitative experiments. Table 6a compares PIF ($\omega = \delta$) to Sliding-PIF with smallest window size ($\omega = \frac{\delta}{20}$), alongside ground truths. Table 6b shows average AUC on 5 runs achieved by PIF and Sliding-PIF at different ω.

As expected, Sliding-PIF performs generally better with smaller window sizes ω (Table 6b), as local models tend to be more accurate when fitted to smaller regions, while large ω do not satisfy the locality principle. An exception is the cookie object, which contains a relatively large anomalous area. In fact, Sliding-PIF performs well when genuine data are the majority in W, thus the best ω for cookie is the second or third-smallest value for planes and quadrics. Spheres, on the contrary, perform best when $\omega = \delta$, since they describe accurately the entire cookie, but worst for small windows as they tend to be biased towards the center of W, struggling to describe

Table 7: (a) Qualitative results of PIF ($\omega = \delta$) and Sliding-PIF ($\omega = \frac{\delta}{20}$), alongside ground truths, when executed on non-smooth objects, alongside average AUC values (b).



borders. Potato has a more ellipsoidal shape, and spheres struggle to describe it for large ω. With medium-sized windows, spheres perform best, showing that locally they best capture potato shape. The higher flexibility of quadrics leads to high variance in the models fitted, resulting in poor performance for medium-sized ω. Carrots are elongated and difficult to approximate with spheres, but they are well described by quadrics and planes, since the local curvature is not very pronounced. Bagels and peaches are difficult for all the model families when ω is large due to their toroidal shape. However, as ω decreases, all families perform better, since the toroidal shape no longer prevails in a small neighborhood.

We further evaluated Sliding-PIF on mechanical objects with non-smooth surfaces—namely *cable gland*, *dowel*, *foam*, *rope*, and *tire*—to explore its limitations (Table 7b). As expected, performance generally declines in this scenario, due to the reduced compatibility with local structural priors. Among these objects, *rope* and *dowel* achieve relatively better performance, as their elongated, near-planar shapes resemble those of smoother objects like *carrot*, and are therefore still amenable to local approximation with simple geometric models (e.g., planes or quadrics). In contrast, *foam* represents a clear worst-case, *Tire* follows closely in difficulty, while *cable gland* lies in between. While the performance remains strong with smooth objects, it declines on highly irregular or non-smooth geometries (*e.g.*, foam, tire), highlighting a key limitation when the model family $\mathcal{F}$ poorly matches the underlying data. Nonetheless, these failure modes are interpretable and reflect the model's transparency. The reasons behind these performance drops can be systematically attributed to three fundamental challenges or limitations of our solution: *i) Violation of local structural assumptions*: in *Foam* local structural regu-

larity is systematically violated. Its highly irregular and non-smooth surfaces cannot be meaningfully captured by geometric primitives, resulting in consistently poor performance across all model families and window sizes. *ii) Mixed surface characteristics*: *Tire* demonstrates the challenges arising when data exhibit spatially inconsistent local properties. While some regions are locally smooth, the presence of treads and sharp geometric transitions results in unstable model fitting, as individual local windows contain fundamentally incompatible structural characteristics that cannot be reconciled by a single geometric model. *iii) Partial structural compatibility*: *Cable gland* exemplifies borderline scenarios for PIF. The combination of both smooth and irregular regions makes genuine data only partially approximable by local models, leading to unpredictable performance due to the spatial heterogeneity of structural properties, where some windows yield reliable models while others fail completely. Regarding model families, results reflect the inherent trade-offs between flexibility and stability. Planes perform well on nearly flat or elongated objects (*e.g.*, *dowel*, *rope*, *carrot*), due to their simplicity and low variance. Quadrics are more expressive and adaptable, but this flexibility results in higher variance across datasets. Spheres tend to perform consistently poor in this non-smooth setting, as their regular geometry rarely aligns with the actual surface structure, despite showing better behavior on smoother objects. These experiments demonstrate the robustness of our approach in complex real-world scenarios, where a global structural prior describing genuine data is unavailable or difficult to define. Sliding-PIF proves effective even when structural assumptions hold only locally, achieving strong performance leveraging a prior based on an appropriate model family $\mathcal{F}$ on windows having size ω. In contrast to global approaches (like PIF), Sliding-PIF is more practical to analyze large-scale and geometrically heterogeneous datasets. Furthermore, the combination of Sliding-PIF and RuzHash-iForest significantly improves runtime and memory efficiency, confirming the scalability of our framework for structure-based anomaly detection.

6. Limitations

The first limitation of our framework is its reliance on structural priors. The method is effective when some prior knowledge about the underlying structures characterizing genuine data is available, and can be expressed through a model family $\mathcal{F}$. While our results on Sliding-PIF show that a local or coarse prior – such as fitting planes to approximate smooth surfaces – can in some cases be sufficient, some minimal understanding of the data's structure remains necessary. For example, it is crucial to know the intrinsic dimensionality of the structures: approximating a 2D surface with a 1D model such as a line would lead to unstable embeddings and poor separation, as the prior would fail to capture the relevant regularities and might instead be overly sensitive to noise.

The second limitation lies in the computational cost of the preference embedding. While the cost of the anomaly scoring stage (Preference Isolation) is mitigated through efficient mechanisms such as LSH partitioning and the sliding window strategy in Sliding-PIF, the construction of the preference space remains computationally intensive, both in terms of runtime and memory. Although Sliding-PIF reduces these demands by restricting computations to local regions, the process is still non-trivial when

scaling to very large datasets. That said, this bottleneck could be addressed in future work through parallelized or GPU-accelerated implementations of the embedding step.

7. Conclusion

Detecting anomalies in structured data is challenging, especially when genuine data lie on low-dimensional manifolds embedded in higher-dimensional ambient spaces. In such settings, traditional methods often fail, as they rely on ambient-space densities, which may not reflect structural inconsistency.

To address this, we introduce PIF, which shifts the perspective from monitoring ambient-space density to structural consistency. Instead of explicitly recovering the underlying models and their inliers –which is often more difficult than detecting anomalies – we construct a preference embedding: each data point is mapped into a high-dimensional space that reflects how well it agrees with a pool of candidate models. In this *preference space*, anomalies naturally appear as isolated points, lacking consensus with the genuine structures.

The modular nature of PIF allows adapting the method to different scenarios:

- When a global structure is assumed and arbitrary distances over preferences are needed, Voronoi-iForest offers a general and accurate solution by leveraging Voronoi tessellations.
- When scalability is crucial, RuzHash-iForest replaces costly distance computations with efficient locality-sensitive hashing. It retains the benefits of structure-based detection while enabling application to large datasets.
- When a locality prior is available, Sliding-PIF allow to perform anomaly detection even even when global structure is hard to define.

As future work, promising directions include integrating non-parametric or learned model families into the embedding process, and designing new distances or hashing schemes tailored to specific structural priors.

Acknowledgements

This work is supported by GEOPRIDE under ID 2022245ZYB and CUP D53D23008370001 (PRIN 2022 M4.C2.1.1 Investment).

Appendix A. RuzHash is a LSH for Ruzicka distance

We present the proof of Theorem 1, introduced in Section 4.3.1, stating that RuzHash is a Locality Sensitive Hashing (LSH) for the Ruzicka distance. The proof builds on Lemma 1, a well-known result stating that MinHash is an LSH for the Jaccard distance. Additionally, Figure A.11 empirically illustrates the correlation between RuzHash and Ruzicka.

Lemma 1 (MinHash)**.** *Given $\boldsymbol{p}, \boldsymbol{q} \in \{0,1\}^m$, let $h^{\boldsymbol{\pi}}_{min} : \{0,1\}^m \to \{1,\dots,m\}$ be the* MinHash *mapping, where $\boldsymbol{\pi} = [\pi_1, \dots, \pi_m]$ is a random permutation of the m dimensions $\{1,\dots,m\}$. It is known [36] that*

$$Pr[h^{\boldsymbol{\pi}}_{min}(\boldsymbol{p}) = h^{\boldsymbol{\pi}}_{min}(\boldsymbol{q})] = \frac{\sum_{\boldsymbol{\pi}} \mathbb{1}(h^{\boldsymbol{\pi}}_{min}(\boldsymbol{p}) = h^{\boldsymbol{\pi}}_{min}(\boldsymbol{q}))}{m!} = \\ = \frac{\sum_{i=1}^m (p_i \wedge q_i)}{\sum_{i=1}^m (p_i \vee q_i)} = \frac{\sum_{i=1}^m \min\{p_i, q_i\}}{\sum_{i=1}^m \max\{p_i, q_i\}} = 1 - d_J(\boldsymbol{p}, \boldsymbol{q}),$$

where $\mathbb{1}$ is the indicator function, $m!$ is the total number of permutations, and $\sum_{\boldsymbol{\pi}}$ sums over all the possible realizations of permutation $\boldsymbol{\pi}$.

Theorem 1 (RuzHash)**.** *Given $\boldsymbol{p}, \boldsymbol{q} \in [0,1]^m$, let $\boldsymbol{\tau} = [\tau_1, \dots, \tau_m]$ be a vector of thresholds where $\tau_i \sim \mathcal{U}_{[0,1)}$, let $\mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau}) = [\mathbb{1}(p_1 > \tau_1), \dots, \mathbb{1}(p_m > \tau_m)] \in \{0,1\}^m$ be the binarization function, and let $h^{\boldsymbol{\pi}}_{min} : \{0,1\}^m \to \{1,\dots,m\}$ be the* MinHash *mapping (Lemma 1). Let* RuzHash *be the function $h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz} = h^{\boldsymbol{\pi}}_{min} \circ \mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau}) : [0,1]^m \to \{1,\dots,m\}$. Then,*

$$Pr[h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\boldsymbol{\tau}}_{ruz}(\boldsymbol{q})] = \frac{\sum_{i=1}^m \min\{p_i, q_i\}}{\sum_{i=1}^m \max\{p_i, q_i\}} = 1 - d_R(\boldsymbol{p}, \boldsymbol{q}). \tag{A.1}$$

Proof. Relying on the definition of RuzHash, it holds that

$$P[h^{\boldsymbol{\pi}}_{min}(\mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau})) = h^{\boldsymbol{\pi}}_{min}(\mathbb{1}(\boldsymbol{q} > \boldsymbol{\tau}))] = \frac{\int_{\boldsymbol{\tau}} \sum_{\boldsymbol{\pi}} \mathbb{1}(h^{\boldsymbol{\pi}}_{min}(\mathbb{1}(\boldsymbol{p} > \boldsymbol{\tau})) = h^{\boldsymbol{\pi}}_{min}(\mathbb{1}(\boldsymbol{q} > \boldsymbol{\tau})))d\boldsymbol{\tau}}{m!} = \\ = \frac{\int_{\boldsymbol{\tau}} \sum_{i=1}^m (\mathbb{1}(p_i > \tau_i) \wedge \mathbb{1}(q_i > \tau_i))d\boldsymbol{\tau}}{\int_{\boldsymbol{\tau}} \sum_{i=1}^m (\mathbb{1}(p_i > \tau_i) \vee \mathbb{1}(q_i > \tau_i))d\boldsymbol{\tau}} = \frac{\int_{\boldsymbol{\tau}} \sum_{i=1}^m \min\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\boldsymbol{\tau}}{\int_{\boldsymbol{\tau}} \sum_{i=1}^m \max\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\boldsymbol{\tau}},$$

where $\int_{\boldsymbol{\tau}}$ integrates over all the possible realizations of $\boldsymbol{\tau}$, and the rest follows from Lemma 1. Since each $\tau_i \sim \mathcal{U}_{[0,1)}$ is an *independent* realization, then

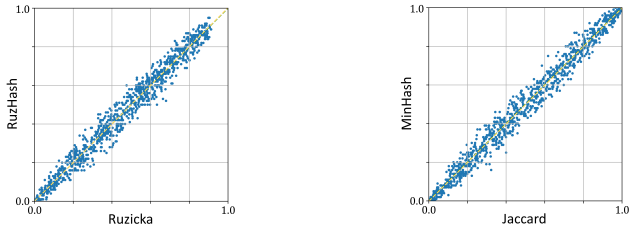
$$\frac{\int_{\boldsymbol{\tau}} \sum_{i=1}^m \min\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\boldsymbol{\tau}}{\int_{\boldsymbol{\tau}} \sum_{i=1}^m \max\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\boldsymbol{\tau}} = \frac{\sum_{i=1}^m \int_0^1 \min\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\tau_i}{\sum_{i=1}^m \int_0^1 \max\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\tau_i}.$$

We have that

$$\min\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\} = \begin{cases} 1 & \text{if } \min\{p_i, q_i\} > \tau_i \\ 0 & \text{otherwise,} \end{cases}$$

and analogously for $\max\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}$. Therefore,

$$\frac{\sum_{i=1}^m \int_0^1 \min\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\tau_i}{\sum_{i=1}^m \int_0^1 \max\{\mathbb{1}(p_i > \tau_i), \mathbb{1}(q_i > \tau_i)\}d\tau_i} = \frac{\sum_{i=1}^m (\int_0^{\min\{p_i,q_i\}} 1\, d\tau_i + \int_{\min\{p_i,q_i\}}^1 0\, d\tau_i)}{\sum_{i=1}^m (\int_0^{\max\{p_i,q_i\}} 1\, d\tau_i + \int_{\max\{p_i,q_i\}}^1 0\, d\tau_i)} = \\ = \frac{\sum_{i=1}^m \int_0^{\min\{p_i,q_i\}} 1\, d\tau_i}{\sum_{i=1}^m \int_0^{\max\{p_i,q_i\}} 1\, d\tau_i} = \frac{\sum_{i=1}^m \min\{p_i, q_i\}}{\sum_{i=1}^m \max\{p_i, q_i\}},$$



(a) (b)

Figure A.11: Correlation between Ruzicka and RuzHash estimated from $n = 1000$ random couples of points $\boldsymbol{p}, \boldsymbol{q} \in [0,1]^m$, where $m = 10000$ and RuzHash is executed $k = 100$ times for each couple, compared to (b) the correlation between Jaccard and MinHash.

and this proves (A.1). □

Figure A.11 shows the correlation between $1 - d_R(\boldsymbol{p}, \boldsymbol{q})$ and RuzHash for $n = 1000$ random couples of points $\boldsymbol{p}, \boldsymbol{q} \in [0,1]^m$, when $m = 10000$ and RuzHash is executed $k = 100$ times for each couple, compared to the correlation between $1 - d_J(\boldsymbol{p}, \boldsymbol{q})$ and MinHash in the same settings.

Appendix B. RuzHash linearly correlates with Ruzicka distance

We prove that, when the RuzHash mapping is modified as described in Section 4.3.2, it maintains a correlation with the Ruzicka distance, with the slope being proportional to the new branching factor b. In Figure B.12, we empirically illustrate the linear correlation for different branching factors.

Theorem 2 ($d_R(\boldsymbol{p}, \boldsymbol{q})$ linearly correlates with b)**.** *Given $\boldsymbol{p}, \boldsymbol{q} \in [0,1]^m$, let $h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}), h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q}) \in \{1, \dots, m\}$ their maps computed by* RuzHash*. Let $\boldsymbol{\beta} = [\beta_1, \dots, \beta_m]$ be a vector in $\{1, \dots, b\}^m$, where each $\beta_i \sim \mathcal{U}_{\{1,\dots,b\}}$. Then,*

$$P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})}] = 1 + \frac{1-b}{b} d_R(\boldsymbol{p}, \boldsymbol{q}). \tag{B.1}$$

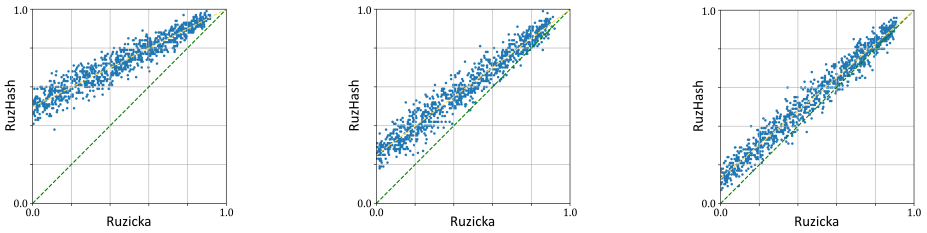
Proof. We split $P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})}]$ in two terms

$$\begin{aligned} P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})}] = {} & P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})} | h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})]\, P[h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})] + \\ & + P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})} | h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) \neq h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})]\, P[h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) \neq h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})]. \end{aligned}$$

We know from Theorem 1 that $P[h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})] = 1 - d_R(\boldsymbol{p}, \boldsymbol{q})$, thus

$$\begin{aligned} P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})}] = {} & P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})} | h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})]\, (1 - d_R(\boldsymbol{p}, \boldsymbol{q})) + \\ & + P[\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})} | h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) \neq h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})]\, d_R(\boldsymbol{p}, \boldsymbol{q}). \end{aligned} \tag{B.2}$$

The probability that $\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p})}$ equals $\beta_{h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})}$ is 1 when $h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{p}) = h^{\boldsymbol{\pi},\tau}_{ruz}(\boldsymbol{q})$, and $\frac{1}{b}$ when



(a) $b = 2$ (b) $b = 4$ (c) $b = 8$

Figure B.12: Correlation between Ruzicka and RuzHash at various b values, estimated from $n = 1000$ random couples of points $\boldsymbol{p}, \boldsymbol{q} \in [0, 1]^m$, when $m = 10000$ and $k = 100$.

$h^{\pi,\tau}_{ruz}(\boldsymbol{p}) \neq h^{\pi,\tau}_{ruz}(\boldsymbol{q})$. Therefore,

$$P[\beta_{h^{\pi,\tau}_{ruz}(\boldsymbol{p})} = \beta_{h^{\pi,\tau}_{ruz}(\boldsymbol{q})}] = 1 - d_R(\boldsymbol{p}, \boldsymbol{q}) + \frac{1}{b} d_R(\boldsymbol{p}, \boldsymbol{q}) = 1 + \frac{1-b}{b} d_R(\boldsymbol{p}, \boldsymbol{q}),$$

and this proves (B.1). □

Corollary 1 ($d_J(\boldsymbol{p}, \boldsymbol{q})$ linearly correlates with b)**.** *Substituting* RuzHash *maps* $h^{\pi,\tau}_{ruz}(\boldsymbol{p}), h^{\pi,\tau}_{ruz}(\boldsymbol{q})$ *with* MinHash *maps* $h^{\pi}_{min}(\boldsymbol{p}), h^{\pi}_{min}(\boldsymbol{p})$ *in Theorem 2, and given Lemma 1, the proof is equivalent. Therefore,*

$$P[\beta_{h^{\pi}_{min}(\boldsymbol{p})} = \beta_{h^{\pi}_{min}(\boldsymbol{q})}] = 1 + \frac{1-b}{b} d_J(\boldsymbol{p}, \boldsymbol{q}).$$

References

[1] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Computing Surveys 41 (3) (2009) 1–58.

[2] P. Bergmann, X. Jin, D. Sattlegger, C. Steger, The MVTec 3D-AD dataset for unsupervised 3D anomaly detection and localization, in: VISIGRAPP, INSTICC, SciTePress, 2022, pp. 202–213.

[3] M. Zhong, M. Lin, C. Zhang, Z. Xu, A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges, Computers & Security 141 (2024) 103821.

[4] P. J. Besl, N. D. McKay, A method for registration of 3-D shapes, TPAMI 14 (2) (1992) 239–256.

[5] R. Brunelli, Template matching techniques in computer vision: theory and practice, J. W. & Sons, 2009.

[6] L. Magri, F. Leveni, G. Boracchi, Multilink: Multi-class structure recovery via agglomerative clustering and model selection, in: CVPR, IEEE, 2021, pp. 1853–1862.

[7] A. Gionis, P. Indyk, R. Motwani, et al., Similarity search in high dimensions via hashing, in: VLDB, Vol. 99, Morgan Kaufmann Publishers, 1999, pp. 518–529.

[8] F. Leveni, L. Magri, G. Boracchi, C. Alippi, PIF: anomaly detection via preference embedding, in: ICPR, IEEE, 2021, pp. 8077–8084.

[9] F. Leveni, L. Magri, C. Alippi, G. Boracchi, Hashing for structure-based anomaly detection, in: ICIAP, Springer Nature, 2023, pp. 25–36.

[10] D. Samariya, A. Thakkar, A comprehensive survey of anomaly detection algorithms, Annals of Data Science 10 (3) (2023) 829–850.

[11] R. K. Halder, M. N. Uddin, M. A. Uddin, S. Aryal, A. Khraisat, Enhancing K-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications, Journal of Big Data 11 (1) (2024) 113.

[12] K. M. Ting, Y. Zhu, M. Carman, Y. Zhu, Z.-H. Zhou, Overcoming key weaknesses of distance-based neighbourhood methods using a data dependent dissimilarity measure, in: SIGKDD, ACM, 2016, pp. 1205–1214.

[13] M. M. Breunig, H.-P. Kriegel, R. T. Ng, J. Sander, LOF: identifying density-based local outliers, in: SIGMOD, ACM, 2000, pp. 93–104.

[14] F. T. Liu, K. M. Ting, Z.-H. Zhou, Isolation-based anomaly detection, TKDD 6 (1) (2012) 1–39.

[15] S. Hariri, M. C. Kind, R. J. Brunner, Extended isolation forest, TKDE 33 (04) (2021) 1479–1489.

[16] J. Lesouple, C. Baudoin, M. Spigai, J.-Y. Tourneret, Generalized isolation forest for anomaly detection, Pattern Recognition Letters 149 (2021) 109–119.

[17] G. Staerman, P. Mozharovskyi, S. Clémençon, F. d'Alché Buc, Functional isolation forest, in: ACML, PMLR, 2019, pp. 332–347.

[18] A. Mensi, D. M. Tax, M. Bicego, Detecting outliers from pairwise proximities: Proximity isolation forests, Pattern Recognition 138 (2023).

[19] A. Mensi, M. Bicego, Enhanced anomaly scores for isolation forests, Pattern Recognition 120 (2021).

[20] H. Xu, Y. Wang, S. Jian, Q. Liao, Y. Wang, G. Pang, Calibrated one-class classification for unsupervised time series anomaly detection, TKDE 36 (11) (2024) 5723–5736.

[21] H. Qiao, H. Tong, B. An, I. King, C. Aggarwal, G. Pang, Deep graph anomaly detection: A survey and new perspectives, TKDE 37 (9) (2025) 5106–5126.

[22] Z. Zuo, Z. Wu, B. Chen, X. Zhong, A reconstruction-based feature adaptation for anomaly detection with self-supervised multi-scale aggregation, in: ICASSP, IEEE, 2024, pp. 5840–5844.

[23] K. Korniszuk, B. Sawicki, Autoencoder-based anomaly detection in network traffic, in: CPEE, IEEE, 2024, pp. 1–4.

[24] Y. Xu, S. Wang, Q. Feng, J. Xia, Y. Li, H.-D. Li, J. Wang, scCAD: Cluster decomposition-based anomaly detection for rare cell identification in single-cell expression data, Nature Communications 15 (1) (2024) 7561.

[25] Q. Zhou, J. Yan, S. He, W. Meng, J. Chen, PointAD: Comprehending 3D anomalies from points and pixels for zero-shot 3D anomaly detection, in: NeurIPS, Vol. 37, Curran Associates, 2024, pp. 84866–84896.

[26] Y. Cheng, Y. Cao, G. Xie, Z. Lu, W. Shen, Towards zero-shot point cloud anomaly detection: A multi-view projection framework, arXiv:2409.13162 (2024).

[27] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning transferable visual models from natural language supervision, in: ICML, Vol. 139, PMLR, 2021, pp. 8748–8763.

[28] M. A. Fischler, R. C. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, CACM 24 (6) (1981) 381–395.

[29] H. Wang, D. Suter, Robust adaptive-scale parametric model estimation for computer vision, TPAMI 26 (11) (2004) 1459–1474.

[30] R. Toldo, A. Fusiello, Robust multiple structures estimation with j-linkage, in: ECCV, Springer Nature, 2008, pp. 537–547.

[31] L. Magri, A. Fusiello, T-linkage: A continuous relaxation of j-linkage for multi-model fitting, in: CVPR, IEEE, 2014, pp. 3954–3961.

[32] P. Jaccard, Étude comparative de la distribution florale dans une portion des alpes et des jura, Bulletin de la SVSN 37 (1901) 547–579.

[33] M. Ružička, Anwendung mathematisch–statisticher methoden in der geobotanik (synthetische bearbeitung von aufnahmen), Biológia 13 (1958) 647.

[34] T. T. Tanimoto, Elementary mathematical theory of classification and prediction, Internal technical report, IBM (1958).

[35] D. E. Knuth, The art of computer programming: Volume 3: Sorting and Searching, Addison-Wesley Professional, 1998.

[36] A. Z. Broder, M. Charikar, A. M. Frieze, M. Mitzenmacher, Min-wise independent permutations, JCSS 60 (3) (2000) 630–659.

[37] Y. Kanazawa, H. Kawakami, Detection of planar regions with uncalibrated stereo using distributions of feature points, in: BMVC, BMVA, 2004, pp. 247–256.